REPASO MODULO 1

La actividad hay que hacerla de manera individual pero presentarla en grupo

Ejercicio 1 – Ver intentos de inicio de sesión SSH

Comando:

```
sudo cat /var/log/auth.log | grep "ssh"
```

Explicación detallada:

- /var/log/auth.log contiene todos los eventos de autenticación: sudo, ssh, inicios de sesión, fallos.
- grep "ssh" filtra solo las líneas relacionadas con SSH.
- Se verán mensajes como:

```
Nov 18 10:45:23 kali sshd[1023]: Failed password for invalid user admin from 192.168.1.50 port 54321 ssh2

Nov 18 10:46:01 kali sshd[1030]: Accepted password for alumno from 192.168.1.34 port 51724 ssh2
```

Interpretación:

- "Failed password": intento fallido, posible fuerza bruta.
- "Accepted password": acceso permitido.
- invalid user indica que intentaron usar usuarios inexistentes.
- from x.x.x.x muestra la IP origen.

Este ejercicio enseña a detectar intrusiones reales.

```
~/Documents/CENEC main ❯ sudo cat /var/log/auth.log | grep "ssh"

2025-11-19T09:16:55.049394+01:00 kali sudo:     kali : TTY=pts/2 ; P
NSCENDED/transcendence ; USER=root ; COMMAND=/usr/bin/systemctl star
2025-11-19T09:16:55.155141+01:00 kali sshd[424666]: Server listening
2025-11-19T09:16:55.156386+01:00 kali sshd[424666]: Server listening
~/Doc/CENEC main ❯
```

🧪 2. ANALIZAR TRÁFICO CON TCPDUMP

Ejercicio 2 – Capturar 50 paquetes

Comando:

```
sudo tcpdump -i eth0 -c 50 -w captura.pcap
```

Explicación detallada:

- -i eth0: selecciona la interfaz (en algunas distros será wlan0 o ens33).
- -c 50: captura solo 50 paquetes y se detiene.
- -w captura.pcap: guarda la captura sin mostrarla en pantalla.

Salida esperada:

```
50 packets captured

52 packets received by filter

0 packets dropped by kernel
```

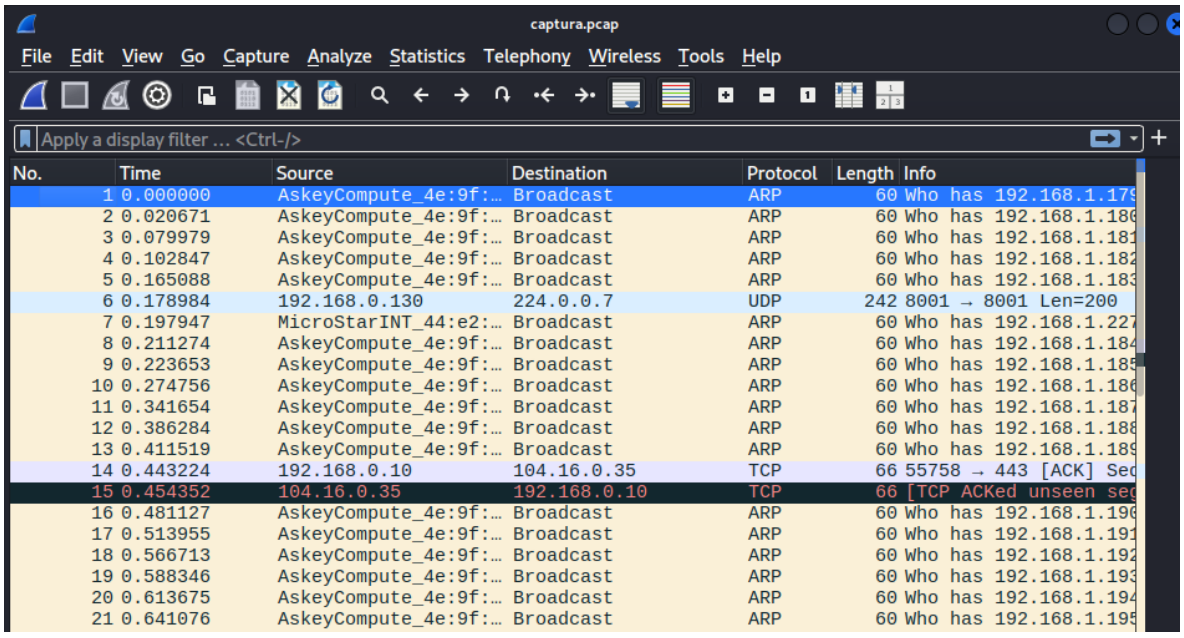
```
~/Documents/box ❯ sudo tcpdump -i eth0 -c 50 -w captura.pcap
tcpdump: listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
50 packets captured
73 packets received by filter
0 packets dropped by kernel
~/Documents/box ❯
```

Después, abrir con Wireshark:

```
wireshark captura.pcap &
```

Qué puede observar el alumno en Wireshark:

- Paquetes ARP, DNS, TCP, UDP.
- Ver los campos: MAC origen, destino, protocolos, puertos, flags TCP.



No.	Time	Source	Destination	Protocol	Length	Info
1	0.000000	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.179
2	0.020671	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.180
3	0.079979	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.181
4	0.102847	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.182
5	0.165088	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.183
6	0.178984	192.168.0.130	224.0.0.7	UDP	242	8001 → 8001 Len=200
7	0.197947	MicroStarINT_44:e2:…	Broadcast	ARP	60	Who has 192.168.1.227
8	0.211274	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.184
9	0.223653	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.185
10	0.274756	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.186
11	0.341654	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.187
12	0.386284	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.188
13	0.411519	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.189
14	0.443224	192.168.0.10	104.16.0.35	TCP	66	55758 → 443 [ACK] Seq
15	0.454352	104.16.0.35	192.168.0.10	TCP	66	[TCP ACKed unseen seg
16	0.481127	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.190
17	0.513955	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.191
18	0.566713	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.192
19	0.588346	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.193
20	0.613675	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.194
21	0.641076	AskeyCompute_4e:9f:…	Broadcast	ARP	60	Who has 192.168.1.195

Ejercicio 3 – Filtrar tráfico DNS

Comando:

```
sudo tcpdump -i eth0 port 53
```

Explicación detallada:

- Port 53 = DNS.
- Muestra peticiones del estilo:

```
IP 192.168.1.25.53412 > 8.8.8.8.53: A? google.com.

IP 8.8.8.8.53 > 192.168.1.25.53412: A 142.250.184.14
```

Interpretación:

- A? google.com. → el cliente pide IP IPv4 de google.com
- A 142.250.184.14 → el servidor responde con la IP

Sirve para detectar malware que usa DNS para comunicarse ("DNS tunneling").

```
~/Documents/box ❯ sudo tcpdump -i eth0 port 53
tcpdump: verbose output suppressed, use -v[v]... for full protocol decode
listening on eth0, link-type EN10MB (Ethernet), snapshot length 262144 bytes
09:25:47.611535 IP 192.168.0.10.38026 > 254.red-80-58-61.staticip.rima-tde.net.domain:
TTPS? cloudflare-dns.com. (36)
09:25:47.612927 IP 192.168.0.10.54295 > 254.red-80-58-61.staticip.rima-tde.net.domain:
? example.org. (29)
09:25:47.613006 IP 192.168.0.10.54295 > 254.red-80-58-61.staticip.rima-tde.net.domain:
AAA? example.org. (29)
09:25:47.615464 IP 192.168.0.10.52211 > 254.red-80-58-61.staticip.rima-tde.net.domain:
? example.org. (29)
09:25:47.620015 IP 192.168.0.10.56436 > 254.red-80-58-61.staticip.rima-tde.net.domain:
? ipv4only.arpa. (31)
09:25:47.620755 IP 192.168.0.10.56436 > 254.red-80-58-61.staticip.rima-tde.net.domain:
AAA? ipv4only.arpa. (31)
09:25:47.630828 IP 254.red-80-58-61.staticip.rima-tde.net.domain > 192.168.0.10.54295:
0/0 A 23.220.75.235, A 23.220.75.238, A 23.215.0.132, A 23.215.0.133 (93)
09:25:47.630828 IP 254.red-80-58-61.staticip.rima-tde.net.domain > 192.168.0.10.54295:
0/0 AAAA 2600:1408:ec00:36::1736:7f2f, AAAA 2600:1406:5e00:6::17ce:bc29, AAAA 2600:140
:17ce:bc3c, AAAA 2600:1408:ec00:36::1736:7f2e (141)
09:25:47.630828 IP 254.red-80-58-61.staticip.rima-tde.net.domain > 192.168.0.10.38026:
0/4 HTTPS (197)
09:25:47.633728 IP 254.red-80-58-61.staticip.rima-tde.net.domain > 192.168.0.10.52211:
0/0 A 23.220.75.235, A 23.220.75.238, A 23.215.0.132, A 23.215.0.133 (93)
09:25:47.638411 IP 254.red-80-58-61.staticip.rima-tde.net.domain > 192.168.0.10.56436:
0/0 A 192.0.0.171, A 192.0.0.170 (63)
09:25:47.639216 IP 254.red-80-58-61.staticip.rima-tde.net.domain > 192.168.0.10.56436:
1/0 (88)
```

🔐 3. CIFRADO Y HASH

Ejercicio 4 – Hash SHA-256

Comandos:

```
echo "hola alumnos" > prueba.txt

sha256sum prueba.txt
```

Salida esperada:

```
5af91d37a95f5f2e7f5ebb1e520986aa01fbf2a580f9c1ce2e67f8df43c2c74a  prueba.txt
```

Explicación detallada:

- SHA-256 siempre devuelve una cadena de 64 caracteres hexadecimales.
- Si añades un solo carácter:

```
echo "x" >> prueba.txt

sha256sum prueba.txt
```

Obtendrás un hash totalmente distinto, demostrando el efecto de avalancha.

```
~/Documents/box ❯ echo "hola alumnos" > prueba.txt
~/Documents/box ❯ cat prueba.txt
hola alumnos
~/Documents/box ❯ sha256sum prueba.txt
6dfc827ba38dd8183068d0286314b6828a95b6ebfb81e72dbd6d286abe7ae678  prueba.txt
~/Documents/box ❯
```

Ejercicio 5 – Cifrar y descifrar con GPG

Comandos:

```
gpg -c secreto.txt
```

- Te pedirá una contraseña de cifrado (simétrica).
- Genera archivo: secreto.txt.gpg.

```
~/Documents/box ❯ gpg -c prueba.txt
~/Documents/box ❯ cat prueba.txt.gpg
⍰
⍰⍰⍰ ⍰c⍰⍰L=⍰⍰⍰⍰Q⍰⍰)1N⍰0⍰⍰X⍰⍰⍰\⍰;⍰'6⍰⍰g|*⍰⍰#]t)&UK⍰⍰lf⍰⍰⍰⍰⍰T⍰
                                                                ^X*4⍰^⍰⍰%
~/Documents/box ❯
```

Descifrar:

```
gpg secreto.txt.gpg
```

Detalle técnico:

- GPG usa AES por defecto en cifrado simétrico.

- -c = cipher / password-based.
- No genera claves públicas/privadas.

```
~/Documents/box ❯ gpg prueba.txt.gpg
gpg: WARNING: no command supplied.  Trying to guess what you mean ...
gpg: AES256.CFB encrypted data
gpg: encrypted with 1 passphrase
File 'prueba.txt' exists. Overwrite? (y/N) n
Enter new filename: new.txt
~/Documents/box ❯ cat new.txt
hola alumnos
~/Documents/box ❯
```

🔄 4. COPIAS DE SEGURIDAD

Ejercicio 6 – Copia con rsync

Comando:

```
sudo rsync -avh /home/ /backup/home/
```

Explicación de parámetros:

- -a: "archive" → copia permisos, dueños, tiempos.
- -v: verbose.
- -h: tamaños legibles.
- /home/ → barra final = copy *content*.
- Si ejecutas otra vez, solo copiará los archivos nuevos.

```
~/Documents/box ❯ sudo rsync -avh /home/kali/Documents/IFCT0109/Cristobal /home/kali/Documents/
CENEC
sending incremental file list
Cristobal/
Cristobal/0-Caso_ciberataque.pdf
Cristobal/1-Ejercicios_comandos_de_linux.pdf
Cristobal/10-Ejercicios_de_dorks_y_shodan.pdf
Cristobal/11-Ejercicios_de_gestión_de_eventos_en_linux.pdf
Cristobal/12-Ejercicios_tar_y_duplicity_en_linux.pdf
Cristobal/13-Ejercicios_arpWatch.pdf
Cristobal/14-Ejercicios_cryptools.pdf
Cristobal/15-Ejercicios_sencillos_de_permisos_en_linux.pdf
Cristobal/16-Ejercicios_john_the_ripper.pdf
Cristobal/17-Ejercicios_con_freeradius.pdf
Cristobal/18-Ejercicios_usando_GPG.pdf
Cristobal/19-Ejercicios_ssh.pdf
Cristobal/2-Ejercicios_comandos_de_linux_2.pdf
Cristobal/20-Ejercicios_sencillos_de_auditoria_y_test_de_intrusión.pdf
Cristobal/21-Ejercicios_usando_tcp_wrapper.pdf
```

```
~/Documents/box ❯ sudo rsync -avh /home/kali/Documents/IFCT0109/Cristobal /home/kali/Documents
CENEC
sending incremental file list

sent 5.33K bytes  received 27 bytes  10.71K bytes/sec
total size is 38.82M  speedup is 7,247.64
```

Salida típica:

```
sending incremental file list

user1/

user1/Documento.txt


sent 1.23K bytes  received 35 bytes  2.53K bytes/sec
```

Ejercicio 7 – Crear imagen comprimida con tar

Comando:

```
sudo tar -czvf backup_home.tar.gz /home
```

Explicación detallada:

- .tar.gz es un contenedor comprimido.
- Sirve como snapshot del momento.

Salida típica:

El terminal mostrará todos los ficheros empaquetados.

```
~/Documents/box ❯ sudo tar -czvf backup_home.tar.gz /home/kali/Documents/CENEC
tar: Removing leading `/' from member names
/home/kali/Documents/CENEC/
/home/kali/Documents/CENEC/18-Ejercicios_usando_GPG.pdf
/home/kali/Documents/CENEC/12-Ejercicios_tar_y_duplicity_en_linux.pdf
/home/kali/Documents/CENEC/35-Ejercicios_de_autopsy.pdf
/home/kali/Documents/CENEC/JAVA/
/home/kali/Documents/CENEC/JAVA/src/
/home/kali/Documents/CENEC/JAVA/src/tema2/
/home/kali/Documents/CENEC/JAVA/src/tema2/Ejercicio1.java
/home/kali/Documents/CENEC/JAVA/src/tema2/EjercicioBucleMenu.java
/home/kali/Documents/CENEC/JAVA/src/tema2/Reto2.java
```

5. NMAP

Ejercicio 8 – Escanear puertos locales

Comando:

```
sudo nmap -sS localhost

Salida típica:

PORT     STATE SERVICE

22/tcp   open  ssh

80/tcp   closed http

631/tcp  open  ipp
```

Interpretación:

- open → un servicio responde.
- closed → el puerto existe, pero no hay aplicación escuchando.
- filtered → firewall bloquea la prueba.

```
~/Documents/box ❯ sudo nmap -sS localhost
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-19 09:32 CET
Nmap scan report for localhost (127.0.0.1)
Host is up (0.0000040s latency).
Other addresses for localhost (not scanned): ::1
Not shown: 994 closed tcp ports (reset)
PORT     STATE SERVICE
22/tcp   open  ssh
1984/tcp open  bigbrother
3000/tcp open  ppp
3306/tcp open  mysql
5432/tcp open  postgresql
8200/tcp open  trivnet1

Nmap done: 1 IP address (1 host up) scanned in 0.10 seconds
```

Ejercicio 9 – Detección de sistema operativo

Comando:

```
sudo nmap -O 192.168.1.1
```

Salida típica:

```
Running: Linux 3.X

OS CPE: cpe:/o:linux:linux_kernel:3

OS details: Linux 3.2 - 4.9
```

Explicación:

- Basado en TTL, opciones TCP, ventanas, timestamp.
- No siempre es 100% exacto.

```
~/Documents/box ❯ sudo nmap -O 192.168.0.1
Starting Nmap 7.95 ( https://nmap.org ) at 2025-11-19 09:41 CET
Nmap scan report for 192.168.0.1
Host is up (0.0010s latency).
Not shown: 995 closed tcp ports (reset)
PORT   STATE    SERVICE
21/tcp filtered ftp
22/tcp filtered ssh
23/tcp filtered telnet
53/tcp filtered domain
80/tcp filtered http
MAC Address: 00:A0:26:D2:68:9A (Teldat)
Warning: OSScan results may be unreliable because we could not f
d port
Device type: broadband router
Running: Teldat embedded
OS details: Teldat K series gateway
Network Distance: 1 hop

OS detection performed. Please report any incorrect results at h
Nmap done: 1 IP address (1 host up) scanned in 0.76 seconds
```

🔥 6. IPTABLES

Ejercicio 10 – Bloquear puerto 23

Comando:

```
sudo iptables -A INPUT -p tcp ---ort 23 -j DROP
```

Explicación:

- DROP no envía respuesta → el atacante cree que no existes.
- Si intentas conectar:

```
telnet localhost 23

Quedará "c"lgado":"Trying 127.0.0.1...
```

(tarda infinito)

```
~/Documents/box ❯ sudo iptables -A INPUT -p tcp --dport 23 -j DROP
~/Documents/box ❯ telnet localhost 23
Trying ::1...
Connection failed: Connection refused
Trying 127.0.0.1...
```

Ejercicio 11 – Permitir solo SSH

Comandos:

```
sudo iptables -P INPUT DROP

sudo iptables -A INPUT -p tcp ---ort 22 -j ACCEPT

sudo iptables -A INPUT -m state ---ate ESTABLISHED,RELATED -j ACCEPT
```

Explicación técnica:

- Política por defecto DROP → todo bloqueado.
- Permitimos puerto 22 (SSH).
- Permitimos conexiones ya establecidas para que webs y DNS funcionen.

```
~/Documents/box ❯ sudo iptables -P INPUT DROP
sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT
sudo iptables -A INPUT -m state --state ESTABLISHED,RELATED -j ACCEPT

~/Documents/box ❯ sudo systemctl start nftables.service
```

Probar:

```
ping go9aceboom
```

→ debe funcionar porque usa conexiones "established".

```
~/Documents/box ❯ ping google.es
PING google.es (142.250.185.3) 56(84) bytes of data.
64 bytes from mad41s11-in-f3.1e100.net (142.250.185.3): icmp_seq=1 ttl=112 time=14.7
64 bytes from mad41s11-in-f3.1e100.net (142.250.185.3): icmp_seq=2 ttl=112 time=12.0
```

🌐 7. PROXY SQUID

Ejercicio 12 – Bloquear Facebook

En squid.conf:

```
acl redes_sociales dstdomain .facebook.com

http_access deny redes_sociales
```

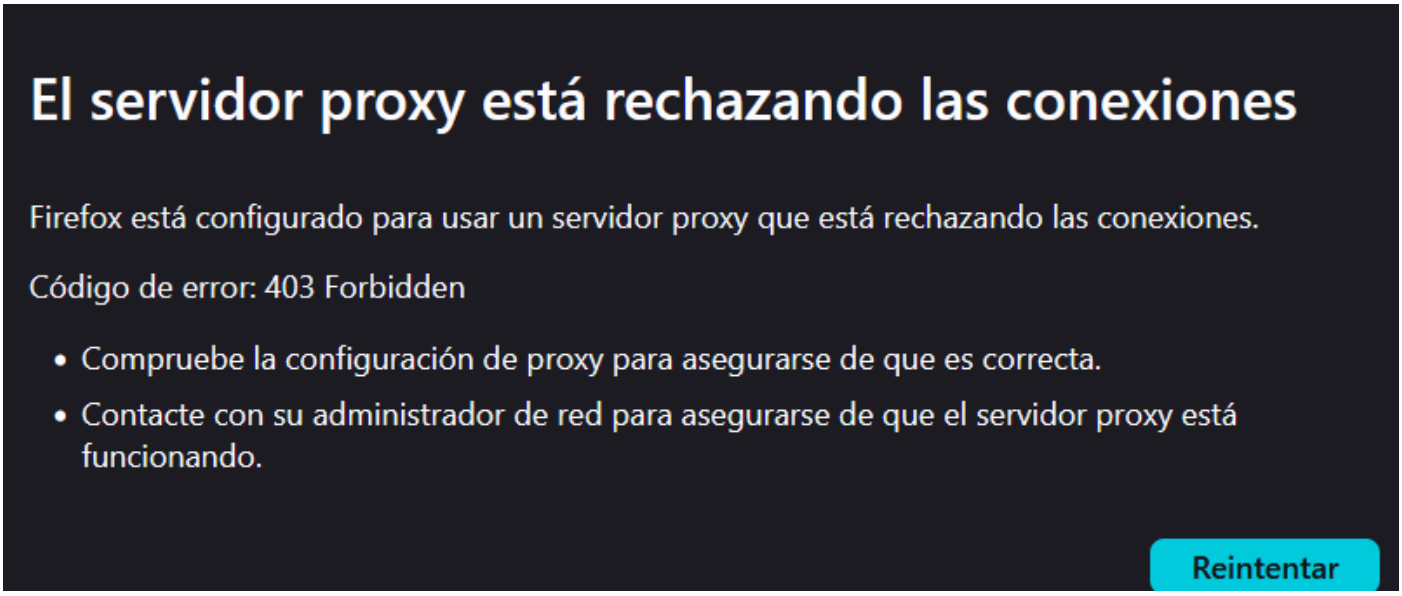
Explicación:

- dstdomain filtra por dominio destino.
- El punto .facebook.com bloquea todos los subdominios.

Comprobación:

En navegador:
→ Acceder a fa10acebookom
Resultado:
"A"cess Denied" "or Squid.

```
~/Documents/box ❯ cat /etc/squid/bloqueadas.txt
.facebook.com
```



Ejercicio 13 – Permitir solo a una red

```
acl red_permitida src 192.168.10.0/24

http_access allow red_permitida

http_access deny all
```

Explicación:

- Las ACL se procesan en orden.
- Si la IP del cliente NO está en 192.168.10.x → queda bloqueado.

Esa línea es de Squid y significa, literalmente:

acl red_permitida src 192.168.10.0/24

La traduzco:

- acl → estás creando una ACL (Access Control List), una lista de control de acceso.
- red_permitida → es el nombre que le das a esa ACL (podría llamarse como quieras).
- src → hace referencia a la IP de origen del cliente (source).
- 192.168.10.0/24 → es la subred de clientes a la que se aplica la ACL (todas las IP desde 192.168.10.1 hasta 192.168.10.25

```
~/Documents/box ❯ cat /etc/squid/reglas.conf
# cargamos el fichero con los dominios que se bloquearan al pasar por el proxy
acl bloqueadas dstdomain "/etc/squid/bloqueadas.txt"
http_access deny bloqueadas

# permitimos acceso a la red de la clase
acl red_local src 192.168.0.0/23
http_access allow red_local


# denegamos todo los demas
http_access deny all
```

```
1763547979.207      3 192.168.0.98 TCP_DENIED/403 3373 CONNECT mail.proton.me:443 - HIER_NONE/- text/html
1763547984.216      1 192.168.0.98 TCP_DENIED/403 3373 CONNECT mail.proton.me:443 - HIER_NONE/- text/html
1763547984.217      1 192.168.0.98 TCP_DENIED/403 3373 CONNECT mail.proton.me:443 - HIER_NONE/- text/html
1763550448.551    154 192.168.0.98 TCP_TUNNEL/200 8059 CONNECT external-content.duckduckgo.com:443 - HIER_DIRECT/52.142.125.222 -
1763550453.264      0 192.168.0.98 TCP_DENIED/403 3369 CONNECT facebook.com:443 - HIER_NONE/- text/html
1763550461.971    135 192.168.0.98 TCP_MISS/301 1050 GET http://google.es/ - HIER_DIRECT/142.251.140.227 text/html
1763550462.484     80 192.168.0.98 TCP_TUNNEL/200 8101 CONNECT www.gstatic.com:443 - HIER_DIRECT/142.250.200.99 -
1763550469.107  21746 192.168.0.98 TCP_TUNNEL/200 152461 CONNECT duckduckgo.com:443 - HIER_DIRECT/52.142.124.215 -
1763550469.514  21900 192.168.0.98 TCP_TUNNEL/200 19949 CONNECT links.duckduckgo.com:443 - HIER_DIRECT/20.223.54.233 -
1763550469.629  21269 192.168.0.98 TCP_TUNNEL/200 10819 CONNECT improving.duckduckgo.com:443 - HIER_DIRECT/52.142.124.215 -
1763550470.552      0 192.168.0.98 TCP_DENIED/403 3377 CONNECT www.facebook.com:443 - HIER_NONE/- text/html
1763550478.547  30153 192.168.0.98 TCP_TUNNEL/200 8345 CONNECT external-content.duckduckgo.com:443 - HIER_DIRECT/52.142.125.222 -
1763550487.943     80 192.168.0.98 TCP_TUNNEL/200 7636 CONNECT encrypted-tbn0.gstatic.com:443 - HIER_DIRECT/142.250.200.110 -
1763550487.947     84 192.168.0.98 TCP_TUNNEL/200 6964 CONNECT encrypted-tbn0.gstatic.com:443 - HIER_DIRECT/142.250.200.110 -
1763550487.948     84 192.168.0.98 TCP_TUNNEL/200 7006 CONNECT encrypted-tbn0.gstatic.com:443 - HIER_DIRECT/142.250.200.110 -
1763550487.948     84 192.168.0.98 TCP_TUNNEL/200 8676 CONNECT encrypted-tbn0.gstatic.com:443 - HIER_DIRECT/142.250.200.110 -
1763550487.960     97 192.168.0.98 TCP_TUNNEL/200 8006 CONNECT encrypted-tbn0.gstatic.com:443 - HIER_DIRECT/142.250.200.110 -
1763550491.123      0 192.168.0.98 TCP_DENIED/403 3377 CONNECT www.facebook.com:443 - HIER_NONE/- text/html
1763550492.101  30120 192.168.0.98 TCP_TUNNEL/200 6861 CONNECT www.google.es:443 - HIER_DIRECT/142.250.200.67 -
```

🔐 8. VPN

Ejercicio 14 – Mostrar interfaces WireGuard

Comando:

```
sudo wg show

Salida típica:

interface: wg0

  public key: Jk2k3k39...

  private key: (hidden)

  listening port: 51820

peer: A9sk3...

  endpoint: 82.153.18.3:50235

 latest handshake: 1 minute ago

  transfer: 2.13 KiB received, 3.89 KiB sent
```

```
~/Documents/box/easy-tls master ❯ sudo wg
interface: wg0
  public key: 0xwovg2x7jbioz542GNPzMul3IUYlwK5+AV1rrm5vGU=
  private key: (hidden)
  listening port: 51820

peer: hwkQuH8ijl7qAlSCHyfx0ROp9txLdGncC07SWT23ZmE=
  endpoint: 192.168.0.131:54819
  allowed ips: 10.10.0.2/32
  latest handshake: 1 minute, 44 seconds ago
  transfer: 1.27 KiB received, 1.12 KiB sent
```

Interpretación:

- "peer" = otro extremo.
- "handshake" = si está activa.
- "transfer" = tráfico cifrado.

```
/mnt/shared ❯ sudo wg-quick up wg0
[#] ip link add dev wg0 type wireguard
[#] wg setconf wg0 /dev/fd/63
[#] ip -4 address add 10.10.0.1/24 dev wg0
[#] ip link set mtu 1420 up dev wg0
```

```
       valid_lft forever preferred_lft forever
kali@kali-virtualbox:~/Documentos$ ping 10.10.0.1
PING 10.10.0.1 (10.10.0.1) 56(84) bytes of data.
64 bytes from 10.10.0.1: icmp_seq=1 ttl=64 time=1.98 ms
64 bytes from 10.10.0.1: icmp_seq=2 ttl=64 time=1.63 ms
64 bytes from 10.10.0.1: icmp_seq=3 ttl=64 time=1.43 ms
64 bytes from 10.10.0.1: icmp_seq=4 ttl=64 time=2.14 ms
64 bytes from 10.10.0.1: icmp_seq=5 ttl=64 time=1.98 ms
64 bytes from 10.10.0.1: icmp_seq=6 ttl=64 time=1.74 ms
^C
--- 10.10.0.1 ping statistics ---
6 packets transmitted, 6 received, 0% packet loss, time 5007ms
rtt min/avg/max/mdev = 1.431/1.815/2.135/0.239 ms
kali@kali-virtualbox:~/Documentos$
```

```
~/Documents/box/easy-tls master ❯ ping 10.10.0.2
PING 10.10.0.2 (10.10.0.2) 56(84) bytes of data.
64 bytes from 10.10.0.2: icmp_seq=1 ttl=64 time=1.11 ms
64 bytes from 10.10.0.2: icmp_seq=2 ttl=64 time=2.24 ms
^C
--- 10.10.0.2 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1040ms
rtt min/avg/max/mdev = 1.114/1.675/2.236/0.561 ms
```

Ejercicio 15 – Conexión OpenVPN

Comando:

```
sudo openvpn --config cliente.ovpn
```

Salida típica:

```
Initialization Sequence Completed
```

Significa:

- Túnel activo.
- Aparece nueva interfaz:

ip a

→ tun0.

```
~/easyrsa ❯ sudo systemctl start openvpn@server
sudo systemctl enable openvpn@server
Created symlink '/etc/systemd/system/multi-user.target.wants/openvpn@server.service' → '/usr/li
b/systemd/system/openvpn@.service'.
```

```
4
2025-11-12 10:51:01 Socket Buffers: R=[212992->212992] S=[212992->212992]
2025-11-12 10:51:01 UDPv4 link local: (not bound)
2025-11-12 10:51:01 UDPv4 link remote: [AF_INET]192.168.0.10:1194
2025-11-12 10:51:01 TLS: Initial packet from [AF_INET]192.168.0.10:1194, sid=f9961acf 1a684052
2025-11-12 10:51:01 VERIFY OK: depth=1, CN=kali
2025-11-12 10:51:01 VERIFY OK: depth=0, CN=kali
2025-11-12 10:51:01 Control Channel: TLSv1.3, cipher TLSv1.3 TLS_AES_256_GCM_SHA384, peer certi
ficate: 2048 bits RSA, signature: RSA-SHA256
2025-11-12 10:51:01 [kali] Peer Connection Initiated with [AF_INET]192.168.0.10:1194
2025-11-12 10:51:01 TLS: move_session: dest=TM_ACTIVE src=TM_INITIAL reinit_src=1
2025-11-12 10:51:01 TLS: tls_multi_process: initial untrusted session promoted to trusted
2025-11-12 10:51:01 PUSH: Received control message: 'PUSH_REPLY,route 10.8.0.1,topology net30,p
ing 10,ping-restart 120,ifconfig 10.8.0.6 10.8.0.5,peer-id 0,cipher AES-256-GCM,protocol-flags
cc-exit tls-ekm dyn-tls-crypt,tun-mtu 1500'
2025-11-12 10:51:01 OPTIONS IMPORT: --ifconfig/up options modified
2025-11-12 10:51:01 OPTIONS IMPORT: route options modified
2025-11-12 10:51:01 OPTIONS IMPORT: tun-mtu set to 1500
2025-11-12 10:51:01 net_route_v4_best_gw query: dst 0.0.0.0
2025-11-12 10:51:01 net_route_v4_best_gw result: via 192.168.0.1 dev enp0s3
2025-11-12 10:51:01 ROUTE_GATEWAY 192.168.0.1/255.255.254.0 IFACE=enp0s3 HWADDR=08:00:27:40:88:
cd
2025-11-12 10:51:02 TUN/TAP device tun0 opened
2025-11-12 10:51:02 net_iface_mtu_set: mtu 1500 for tun0
2025-11-12 10:51:02 net_iface_up: set tun0 up
2025-11-12 10:51:02 net_addr_ptp_v4_add: 10.8.0.6 peer 10.8.0.5 dev tun0
2025-11-12 10:51:02 net_route_v4_add: 10.8.0.1/32 via 10.8.0.5 dev [NULL] table 0 metric -1
2025-11-12 10:51:02 Initialization Sequence Completed
2025-11-12 10:51:02 Data Channel: cipher 'AES-256-GCM', peer-id: 0
2025-11-12 10:51:02 Timers: ping 10, ping-restart 120
2025-11-12 10:51:02 Protocol options: protocol-flags cc-exit tls-ekm dyn-tls-crypt
```

```
root@kali-virtualbox:/mnt/shared/HACER# ping 10.8.0.1
PING 10.8.0.1 (10.8.0.1) 56(84) bytes of data.
64 bytes from 10.8.0.1: icmp_seq=1 ttl=64 time=1.38 ms
64 bytes from 10.8.0.1: icmp_seq=2 ttl=64 time=1.28 ms
64 bytes from 10.8.0.1: icmp_seq=3 ttl=64 time=1.68 ms
64 bytes from 10.8.0.1: icmp_seq=4 ttl=64 time=1.22 ms
64 bytes from 10.8.0.1: icmp_seq=5 ttl=64 time=1.27 ms
^C
--- 10.8.0.1 ping statistics ---
5 packets transmitted, 5 received, 0% packet loss, time 4005ms
rtt min/avg/max/mdev = 1.215/1.367/1.684/0.167 ms
root@kali-virtualbox:/mnt/shared/HACER#
```

🧪 9. FORENSE

Ejercicio 16 – Listar archivos de una imagen

Comando:

orma parte de The Sleuth Kit (TSK) y se utiliza en análisis forense digital para listar el contenido de un sistema de archivos dentro de una imagen de disco, incluyendo archivos borrados.

Aquí tienes una explicación clara, detallada y práctica:

🔍 ¿Qué hace fls?

fls examina una imagen forense (por ejemplo, imagen.dd, imagen.E01, particion.raw, etc.) y muestra:

- directorios y archivos existentes
- archivos borrados pero recuperables
- metadatos (inodos)
- estructura interna del sistema de ficheros

Es una herramienta esencial en investigaciones para ver "qué había" sin alterar la evidencia.

```
fls imagen.dd

Salida típica:

d/d 11: lost+found

r/r 12: documento.doc

r/r 13: foto.jpg
```

Significado:

- d/d: directorio.
- r/r: archivo regular.
- Muestra incluso archivos borrados que siguen en disco.

```
~/Documents/box ❯ fls lab.img
d/d 11: lost+found
d/d 13: docs
V/V 25585:      $OrphanFiles
```

---

10. WLAN – AIRCRACK-NG

Ejercicio 17 – Modo monitor

El modo monitor (monitor mode) es un modo especial de funcionamiento de las tarjetas Wi-Fi que permite capturar todo el tráfico inalámbrico que se transmite en un canal, aunque ese tráfico no vaya dirigido a tu equipo.

Comando:

```
sudo airmon-ng start wlan0
```

Salida:

monitor mode enabled on wlan0mon

Explicación:

- Desactiva Network Manager para evitar conflictos.
- Permite capturar paquetes 802.11.

Ejercicio 18 – Escanear redes

Comando:

```
sudo airodump-ng wlan0mon

Salida típica:

BSSID              PWR  Beacons  #Data  CH  ENC   ESSID

00:14:6C:7E:40:80  -45       10     50  6   WPA2  AulaInf
```

Interpretación:

- BSSID = MAC del AP.
- PWR = potencia (más alto = más cerca).
- ENC = cifrado (WPA2, WPA3).
- #Data = tráfico capturado.

🧹 11. BORRADO SEGURO

Ejercicio 19 – shred

shred es una herramienta diseñada para evitar que un archivo eliminado pueda ser recuperado con técnicas forenses, como:

- herramientas de recuperación de datos,
- análisis de disco,
- reconstrucción de bloques eliminados, etc.

En lugar de simplemente borrar el archivo, shred lo sobrescribe varias veces antes de eliminarlo definitivamente.

Comando:

```
shred -u secreto.txt
```

Explicación detallada:

- Sobrescribe varias veces con datos aleatorios.
- -u = borra al final.
- Mucho más seguro que un simples rm.

```
~/Documents/box ❯ shred -u prueba.txt
~/Documents/box ❯
```

🔍 12. SERVICIOS Y PUERTOS

Ejercicio 20 – Ver puertos en escucha

Comando:

es una herramienta muy útil en Linux para ver qué puertos están en escucha y qué procesos están usando esos puertos.

A continuación te explico qué significa, para qué sirve y cómo interpretar su salida.

🧠 ¿Qué es ss?

ss (socket statistics) es el reemplazo moderno de netstat.
Es más rápido, más preciso y soporta más información.

```
sudo ss -tulnp

Salida:

tcp   LISTEN 0  128  0.0.0.0:22    0.0.0.0:*   users:(("sshd",pid=901,fd=3))

udp   LISTEN 0  128  0.0.0.0:53    0.0.0.0:*   users:(("dnsmasq",pid=560,fd=5))
```

Interpretación:

- TCP puerto 22 → SSH corriendo.
- UDP 53 → DNS local activo.
- "pid" = proceso propietario.

```
~/Documents/box ❯ sudo ss -tulnp                                                    system
Netid    State      Recv-Q     Send-Q         Local Address:Port          Peer Address:Port
Process
udp      UNCONN     0          0                  0.0.0.0:1194               0.0.0.0:*
 users:(("openvpn",pid=756,fd=6))
udp      UNCONN     0          0                127.0.0.1:161                0.0.0.0:*
 users:(("snmpd",pid=215751,fd=6))
udp      UNCONN     0          0                   [::1]:161                   [::]:*
 users:(("snmpd",pid=215751,fd=7))
tcp      LISTEN     0          512                0.0.0.0:1984               0.0.0.0:*
 users:(("xymond",pid=209928,fd=3))
tcp      LISTEN     0          4096             127.0.0.1:33055              0.0.0.0:*
 users:(("containerd",pid=750,fd=10))
tcp      LISTEN     0          200              127.0.0.1:5432               0.0.0.0:*
 users:(("postgres",pid=815,fd=7))
tcp      LISTEN     0          4096               0.0.0.0:443                0.0.0.0:*
 users:(("docker-proxy",pid=452413,fd=4))
tcp      LISTEN     0          128                0.0.0.0:22                 0.0.0.0:*
```